

 Core Relational Joins

INSTANT

 Self Join & UNION Sets Data Analysis Diploma • Session 27

SQL

JOINS & SUBQUERIES

SQL Joins · Self Join · Cross Join · UNION · Subqueries · EXISTS & IN · Ranking Functions

4 Blocks**Relational Pillars**

Joins, Sets, Subqueries &
Ranking

31 Slides**Interactive Guide**

Multi-Table Analytics

5 Lab Tasks**Hands-on Queries**

Real Business Data

Start Session 27 → Subqueries & Correlated Logic Window Ranking Functions

— MULTI-TABLE QUERY MASTERY

Session Agenda

By the end of this session you will write multi-table analytical queries — the core skill behind every real dashboard.

1

Core SQL Joins

INNER, LEFT, RIGHT, FULL OUTER — combining data across related tables.

2

Self Join, Cross Join & UNION

A table joined to itself, every-row combinations, and stacking result sets.

3

Subqueries

Queries inside queries — WHERE, IN, EXISTS, and correlated subqueries.

4

Ranking Functions

ROW_NUMBER, RANK, DENSE_RANK, NTILE with OVER() and PARTITION BY.



Outcome: By the end of this session you will write multi-table analytical queries — the core skill behind every real dashboard and enterprise report.

— RECAP: SINGLE TABLE TO MULTI-TABLE

Quick Recap — Sessions 2 & 3

You already know how to filter, sort, and label a single table — but real business answers require connecting related tables:

YOU ALREADY KNOW HOW TO FILTER, SORT, AND LABEL A SINGLE TABLE:

 **WHERE** filters rows by a condition

 **ORDER BY** sorts the result set

 **CASE WHEN** labels rows conditionally

 **Aggregate functions** summarize groups

ALL FROM ONE TABLE: STUDENTS

 Session 3 Query Style: One Table Only

Students Table

```
SELECT
  StudentName,
  City,
  CASE
    WHEN Status = 'Active' THEN 'Currently Studying'
    ELSE 'Inactive'
  END AS StudentState
FROM Students
WHERE City = 'Cairo'
ORDER BY StudentName ASC;
```

● Limited to fields existing physically inside the Students table

 **The Limit:** A single table can't tell you which Course a student enrolled in, or which Campaign brought in a Lead. For that, we need **JOINS**!

— NORMALIZATION & LOOKUPS

Why Do We Need Joins?

In Excel you'd use VLOOKUP or INDEX/MATCH. In relational databases, normalized tables connect via shared foreign keys.

Think Excel First

In Excel, you'd use **VLOOKUP** or **INDEX/MATCH** to pull a value from another sheet — like looking up a CourseName from a Courses sheet using a CourseID.

SQL Databases Work The Same Way

Data lives in separate, normalized tables to avoid repetition. A **JOIN** is how SQL performs that lookup — matching rows across tables using a shared key column.

Excel Analogy:

VLOOKUP(CourseID, Courses!A:B, 2, FALSE) ≈ SQL JOIN ON CourseID

SHARED KEY RELATIONSHIP

Enrollments

Foreign Key

EnrollmentID

StudentID

CourseID (FK)

EnrollDate



matched on CourseID

Courses

Primary Key

CourseID (PK)

CourseName

Category

Price



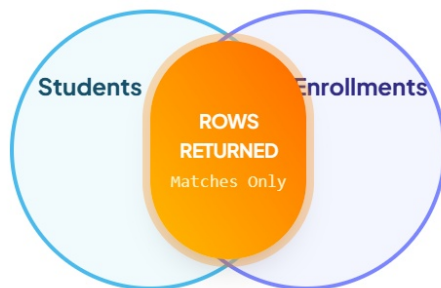
Key takeaway: Normalization keeps data clean and small. Joins are the price — and the superpower — of relational database design.

— STRICT MATCHING ONLY

INNER JOIN — Only the Matches

Returns only rows where the join condition matches in BOTH tables. Students without enrollments disappear.

VENN DIAGRAM REPRESENTATION



Unmatched students & unmatched enrollments are excluded.

What it does:

- Returns only rows where the join condition matches in BOTH tables.
- If a StudentID has no row in Enrollments, that student disappears completely from the query output.
- **The most common join** — use it whenever you only care about strictly connected records.
- 💡 **Default:** When you write just **JOIN** without a modifier, SQL Server treats it as **INNER JOIN**.



Plain English: “Show me every enrollment record, together with the student’s name and course — but only for students who actually enrolled.”

— SELECT · FROM · INNER JOIN · ON

INNER JOIN — Syntax

The ON clause defines HOW tables connect (the key match). WHERE filters the joined result further.

</> T-SQL INNER JOIN Syntax

Canonical Pattern

```
SELECT columns
FROM TableA
INNER JOIN TableB
    ON TableA.KeyColumn = TableB.KeyColumn;
```

1

Select Columns

Choose the columns you want to see from either table.

2

Start FROM Main Table

The left-hand table acts as the primary starting point.

3

INNER JOIN & ON

Specify the second table and match key columns in the ON clause.



ON vs WHERE: The **ON** clause defines **HOW** tables connect (the key match condition). The **WHERE** clause (optional, added afterwards) filters the joined result set further — e.g. **WHERE City = 'Cairo'**.

3-TABLE RELATIONAL QUERY

INNER JOIN — Worked Example

Goal: list every student together with the course name they enrolled in across Students, Enrollments, and Courses.

</> Goal: List Every Student with Enrolled Course Name

Students + Enrollments + Courses

```
SELECT
    s.StudentName,
    c.CourseName,
    e.EnrollDate
FROM Students AS s
INNER JOIN Enrollments AS e ON s.StudentID = e.StudentID
INNER JOIN Courses AS c      ON e.CourseID = c.CourseID;
```

Joined Query Output

3 rows matched

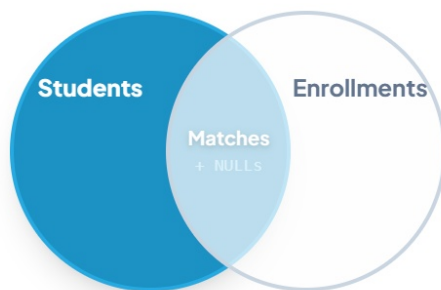
STUDENTNAME	COURSENAME	ENROLLDATE
Mona Adel	SQL for Analysts	2026-02-11
Ali Hassan	Excel Fundamentals	2026-02-14
Yara Fathy	Python Basics	2026-03-02

— ALL ROWS FROM PRIMARY TABLE

LEFT JOIN — Keep Everything on the Left

Returns ALL rows from the left table, plus matches from the right table. Missing values return as NULL.

LEFT OUTER JOIN DIAGRAM



100% of Left Rows Preserved!

What it does:

- Returns ALL rows from the left table, plus any matching rows from the right table.
- If there is no match on the right, its columns come back as **NULL**.
- Perfect for finding "who is missing": e.g. registered students with zero course enrollments.

```
SELECT s.StudentName, e.EnrollmentID FROM Students s LEFT JOIN Enrollments e ON
s.StudentID = e.StudentID;
```

— DATA AUDITING & NULL FILTERING

LEFT JOIN — Finding Students With Zero Enrollments

The LEFT JOIN + WHERE IS NULL pattern: finding non-purchasers, unassigned leads, and courses without students.

</> Finding Inactive Students (The Anti-Join Pattern)

WHERE e.EnrollmentID IS NULL

```
SELECT
    s.StudentID,
    s.StudentName
FROM Students AS s
LEFT JOIN Enrollments AS e ON s.StudentID = e.StudentID
WHERE e.EnrollmentID IS NULL;

-- IS NULL = 'no matching row was found on the right side'
```

✓ THE LEFT JOIN + IS NULL PATTERN

This is one of the most useful diagnostic patterns in SQL: join to the related table, then filter for the rows where nothing matched (**IS NULL**). It directly answers critical business questions:

● "Which students haven't paid?"

● "Which course has no attendance?"

● "Which campaign generated zero leads?"

— MIRRORING LEFT JOIN

RIGHT JOIN — Keep Everything on the Right

RIGHT JOIN mirrors LEFT JOIN — in practice, analysts mostly rewrite RIGHT JOIN as LEFT JOIN by swapping table order.

RIGHT OUTER JOIN



💡 Industry Practice:

Analysts rarely write RIGHT JOIN — they simply swap the table positions and write a **LEFT JOIN** instead. It reads naturally from top-to-bottom and left-to-right.

</> Finding Agents With Zero Assigned Leads

Leads RIGHT JOIN SalesAgents

```
-- Every sales agent, even those with zero leads yet:
SELECT
    a.AgentName,
    l.LeadName
FROM Leads AS l
RIGHT JOIN SalesAgents AS a ON l.AgentID = a.AgentID
WHERE l.LeadID IS NULL;
```

-- Result: returns agents not yet assigned any lead

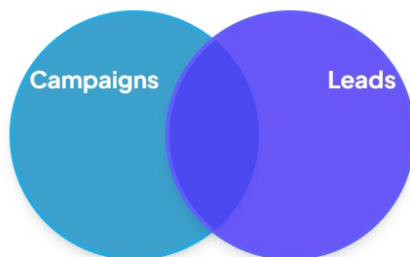
● Swapping tables to 'SalesAgents LEFT JOIN Leads' yields identical results

— UNION OF BOTH DATASETS

FULL OUTER JOIN — Keep Everything, Both Sides

FULL OUTER JOIN keeps every row from BOTH tables. Unmatched columns from either side return as NULL.

FULL OUTER JOIN



FULL OUTER JOIN keeps every row from BOTH tables. Unmatched columns from either side return as NULL.

</> Full Relationship Audit

Campaigns + Leads

```
SELECT
    c.CampaignName,
    l.LeadName
FROM Campaigns AS c
FULL OUTER JOIN Leads AS l ON c.CampaignID = l.CampaignID;
```

```
-- Rows with NULL LeadName: campaign got no leads
-- Rows with NULL CampaignName: lead has no valid campaign
```

● Perfect for complete end-to-end data reconciliation audits

🛡️ **Use Case:** Audit data quality — find orphaned records on both ends in a single query (campaigns with 0 leads AND leads that reference a deleted or missing campaign).

— CHEAT SHEET & SELECTION RULES

Joins Comparison — Choosing the Right One

Rule of thumb: start with INNER JOIN. Switch to LEFT JOIN the moment you need to ask "who or what is missing?".

INNER JOIN

STRICT

RETURNS:

Only matching rows from both tables

USE WHEN:

Enrollments with valid student & course records

LEFT JOIN

MOST USED

RETURNS:

All left rows + matches from right

USE WHEN:

Students who registered but never enrolled

RIGHT JOIN

SWAPPABLE

RETURNS:

All right rows + matches from left

USE WHEN:

Agents with no assigned leads yet

FULL OUTER

AUDIT

RETURNS:

All rows from both, matched or not

USE WHEN:

Full data audit of campaigns vs leads



Rule of Thumb: Start with **INNER JOIN**. Switch to **LEFT JOIN** the moment you need to ask "who or what is missing?".

— REAL BUSINESS DATA FLOW

Putting It Together — A Four-Table Join

Track the full student lifecycle in one query: Students → Enrollments → Courses → Payments.

</> Full Student Lifecycle Query

4 Tables

```
SELECT
  s.StudentName,
  c.CourseName,
  e.EnrollDate,
  p.Amount,
  p.Status AS PaymentStatus
FROM Students AS s
INNER JOIN Enrollments AS e ON s.StudentID = e.StudentID
INNER JOIN Courses AS c      ON e.CourseID  = c.CourseID
LEFT JOIN Payments AS p      ON e.EnrollmentID = p.EnrollmentID
ORDER BY s.StudentName;
```

● Multiplies dimensions horizontally without duplicating student rows

DATA PIPELINE STEP-BY-STEP

Students

who is enrolling

↓ INNER JOIN

+ Enrollments

in which course

↓ INNER JOIN

+ Courses

course details

↓ LEFT JOIN

+ Payments

did they pay?



Critical Architecture Decision: Payments uses **LEFT JOIN** — because a student may have enrolled without paying yet. An INNER JOIN would have accidentally hidden unpaid enrollments from your enrollment report!

— SAME TABLE ALIASED TWICE

SELF JOIN — A Table Joined to Itself

SQL needs two distinct aliases (a1 and a2) to compare 'this row' against other rows in the exact same physical table.

SalesAgents (as a1)

Alias 1

AgentID: 101

AgentName: Nourhan Adel

Region: Cairo

Represents the first agent in comparison

SalesAgents (as a2)

Alias 2

AgentID: 102

AgentName: Tarek Sami

Region: Cairo

Same physical table, aliased independently

🔗 Why alias twice?

SQL needs two distinct table aliases to tell “**this row**” apart from “**the row I'm comparing it to**”. We give the same physical table two aliases (like **a1** and **a2**) and join them on a condition that relates rows to each other — for example, matching agents working in the same region.

— DEDUPLICATION WITH <

SELF JOIN — Worked Example

Pair up sales agents working in the same region — using `a1.AgentID < a2.AgentID` to prevent self-pairing and duplicates.

</> Peer Benchmarking in the Same Region

SalesAgents SELF JOIN

```
SELECT
    a1.AgentName AS Agent1,
    a2.AgentName AS Agent2,
    a1.Region
FROM SalesAgents AS a1
INNER JOIN SalesAgents AS a2
    ON a1.Region = a2.Region
    AND a1.AgentID < a2.AgentID;

-- a1.AgentID < a2.AgentID avoids pairing an agent with themselves,
-- and avoids listing each pair twice (A-B and B-A)
```

 **Why the < condition matters:** Without it, every agent would match themselves (**Region = Region**), and every pair would appear twice (e.g. Ali & Mona, then Mona & Ali). This filtering trick (**a1.AgentID < a2.AgentID**) is unique to self joins and guarantees clean, distinct pairings.

M × N COMBINATIONS

CROSS JOIN — Every Row × Every Row

A CROSS JOIN pairs every row of Table A with every row of Table B without any ON condition — handle with care!

CARTESIAN PRODUCT (3 × 3 = 9)

Course A Jan	Course A Feb	Course A Mar
Course B Jan	Course B Feb	Course B Mar
Course C Jan	Course C Feb	Course C Mar

3 courses × 3 months = 9 generated rows

CROSS JOIN Syntax

No ON Clause

```
SELECT *
FROM Courses
CROSS JOIN (
    VALUES ('Jan'), ('Feb'), ('Mar')
) AS Months(MonthName);

-- No ON clause – every combination is produced automatically.
```

Multiplies Table A row count by Table B row count



Handle with care: 1,000 rows **CROSS JOIN** 1,000 rows = **1,000,000 rows!** Always know your row counts before running a CROSS JOIN on real enterprise tables to prevent memory explosion.

CALENDAR & AVAILABILITY GRIDS

CROSS JOIN — Building a Schedule Matrix

Generate a full course-availability grid: pairing every course with every quarter of the year instantly.

</> Goal: Generate Course Availability Matrix for All 4 Quarters

Courses × Quarters

```
SELECT c.CourseName, q.QuarterName
FROM Courses AS c
CROSS JOIN ( VALUES ('Q1'), ('Q2'), ('Q3'), ('Q4') ) AS q(QuarterName)
ORDER BY c.CourseName, q.QuarterName;
```

Generated Schedule Matrix

2 courses × 4 quarters = 8 slots

COURSE \ QUARTER	Q1	Q2	Q3	Q4
SQL for Analysts	Slot Ready	Slot Ready	Slot Ready	Slot Ready
Excel Fundamentals	Slot Ready	Slot Ready	Slot Ready	Slot Ready

Instant availability grid created with zero manual row insertion.

— HORIZONTAL VS VERTICAL COMBINING

UNION vs UNION ALL — Stacking Result Sets

Joins combine tables side-by-side (more columns). UNION stacks result sets on top of each other (more rows).

Joins combine tables side-by-side (more columns). UNION stacks result sets on top of each other (more rows).

UNION

Deduplicated

Removes duplicate rows automatically:

```
SELECT Email FROM Students
UNION
SELECT Email FROM Leads;
```

⚡ Slower (runs an extra sorting/deduplication step) but guarantees a unique list.

UNION ALL

Includes Duplicates

Keeps every row, including duplicates:

```
SELECT Email FROM Students
UNION ALL
SELECT Email FROM Leads;
```

🚀 Faster — use whenever duplicates are acceptable or known to be impossible.

— MARKETING & STUDENT OUTREACH

UNION — Building a Unified Contact List

The golden rule of UNION: matching column count, order, compatible types, and column names derived from the first SELECT.

</> Unified Contact Directory (Students + Leads)

UNION Set Operation

```
SELECT StudentName AS FullName, Email, 'Student' AS SourceType
FROM Students
```

```
UNION
```

```
SELECT LeadName AS FullName, Email, 'Lead' AS SourceType
FROM Leads;
```



The Golden Rule of UNION: Both SELECT statements must return the exact same number of columns, in compatible data types and positions. Column headers in the final output are determined entirely by the **FIRST SELECT statement** (which is why we aliased StudentName to **FullName**).

— INNER RUNS FIRST · PASSES TO OUTER

Subqueries — A Query Inside a Query

A subquery is a nested SELECT inside WHERE, SELECT, or FROM. Two flavors: Scalar (single value) vs Multi-row (list).

</> OUTER & INNER QUERY EXECUTION

```
-- OUTER QUERY
SELECT CourseName FROM Courses
WHERE Price > (
  INNER QUERY (RUNS FIRST!)
  SELECT AVG(Price) FROM Courses
);
```

Inner query resolves to a value first, which is immediately supplied to the outer query filter.

Two Core Flavors:

Scalar Subquery

Returns exactly ONE value (1 row, 1 column) — usable with =, >, <, >=, <=.

Multi-row Subquery

Returns a LIST of values — usable with IN, EXISTS, ANY, ALL.

Where can it live? Inside WHERE (most common), inside SELECT, or inside FROM.

— COMPUTED SINGLE VALUE FILTER

Scalar Subquery in WHERE

Goal: find every course priced above the average price across all courses using `WHERE Price > (SELECT AVG(Price) ...)`.

</> Goal: Find Every Course Priced Above the Overall Average

Scalar Subquery

```
SELECT CourseName, Price
FROM Courses
WHERE Price > (SELECT AVG(Price) FROM Courses);
```

1

Inner Query Runs Once

`SELECT AVG(Price)` produces a single number, e.g. 1,850.

2

Replaces Subquery

The query simplifies to: `WHERE Price > 1850`.

3

Outer Query Filters

Every course with price above 1,850 is returned cleanly in the result.



Cardinal Rule: A scalar subquery used with standard comparison operators (`>`, `<`, `=`) must return **exactly one row and one column**. If the inner query returns multiple rows, SQL Server throws an error!

— DYNAMIC SET COMPARISON

IN + Subquery — Matching Against a List

Find every student who enrolled in a course from the "Data Analytics" category — works like Excel MATCH against a range.

</> Goal: Find Students in the 'Data Analytics' Category

IN + Subquery

```
SELECT StudentName
FROM Students
WHERE StudentID IN (
    SELECT e.StudentID
    FROM Enrollments AS e
    INNER JOIN Courses AS c ON e.CourseID = c.CourseID
    WHERE c.Category = 'Data Analytics'
);
```



How IN works: The inner query returns a **LIST of StudentIDs**. The outer WHERE keeps only rows whose StudentID appears anywhere in that list — just like Excel's **MATCH** against a range.

— HIGH-PERFORMANCE CORRELATED FILTER

EXISTS — Checking for Existence

EXISTS asks a simple yes/no question: 'does at least one matching row exist?' It never inspects the actual data values.

</> Checking Existence with EXISTS

Correlated EXISTS

```
SELECT StudentName
FROM Students AS s
WHERE EXISTS (
    SELECT 1
    FROM Enrollments AS e
    WHERE e.StudentID = s.StudentID
);
```

-- 'SELECT 1' is a convention: EXISTS only cares whether a row is found



Notice the Correlated Reference: `e.StudentID = s.StudentID` connects the inner query back to the **CURRENT outer row**. This inner query re-runs once per outer row, answering a simple Yes/No boolean question.

— NULL HAZARDS & SHORT-CIRCUITING

EXISTS vs IN — Which One to Use

Rule of thumb: use IN for short, simple static lists. Use EXISTS for correlated checks and large datasets.

IN

- Compares a column against a fixed list of values.
- Simple, intuitive, and readable for small static lists.
- **NULL Hazard:** If the subquery contains NULLs, `NOT IN` can silently return zero rows!

Best for small, explicit lists

EXISTS

- Checks only whether a matching row exists — ignores actual values.
- **High Performance:** The SQL engine stops scanning immediately at the first match.
- The standard choice for correlated checks and large enterprise tables.

Best for correlated subqueries & large tables



Rule of Thumb: Use `IN` for short, simple static lists. Use `EXISTS` for correlated checks and larger datasets.

GROUP BENCHMARK PER ROW

Correlated Subquery — Above-Average Payments

Inner query references a column from the outer row (e.CourseID), re-running per row to compare against its own course average.

</> Goal: Find Payments Above Average for That Same Student's Course

Correlation on e.CourseID

```
SELECT
    p.PaymentID,
    e.CourseID,
    p.Amount
FROM Payments AS p
INNER JOIN Enrollments AS e ON p.EnrollmentID = e.EnrollmentID
WHERE p.Amount > (
    SELECT AVG(p2.Amount)
    FROM Payments AS p2
    INNER JOIN Enrollments AS e2 ON p2.EnrollmentID = e2.EnrollmentID
    WHERE e2.CourseID = e.CourseID -- correlation: ties inner query to outer row
);
```



Correlated vs Simple Subquery: A simple subquery runs once. A correlated subquery re-runs for every row of the outer query, because it references an outer column (e.CourseID). This lets you compare each row against **ITS OWN group average!**

— OVER() · PARTITION BY · ORDER BY

Ranking Functions — ROW_NUMBER, RANK, DENSE_RANK, NTILE

Number rows within a result set — essential for leaderboards, top-N reporting, and percentile customer bucketing.

```
<FUNCTION>() OVER ( PARTITION BY <group_column> ORDER BY <sort_column> DESC )
```

ROW_NUMBER()

UNIQUE

Unique sequential number. Ties get different numbers (1, 2, 3, 4...).

1, 2, 3, 4

RANK()

SKIPS

Ties share a rank; the next rank skips ahead (1, 2, 2, 4...).

1, 2, 2, 4

DENSE_RANK()

NO SKIP

Ties share a rank; the next rank does NOT skip (1, 2, 2, 3...).

1, 2, 2, 3

NTILE(n)

BUCKETS

Splits rows into n equal-sized buckets — e.g. NTILE(4) for quartiles.

Bucket 1..4



PARTITION BY: Restarts the ranking for each group (e.g. each Region separately) — like a subtotal that resets. Omit it to rank the entire table as one single group.

PARTITION RESET & TIE BEHAVIOR

Ranking Sales Agents by Commission, Per Region

Notice the two tied Cairo agents at 38,500 both receive rank 2 — and the next rank skips directly to 4.

Rank Agents by Commission Per Region

SalesAgents

```
SELECT
  AgentName,
  Region,
  Commission,
  RANK() OVER (
    PARTITION BY Region
    ORDER BY Commission DESC
  ) AS RegionRank
FROM SalesAgents;
```

Resets ranking to 1 at each new Region boundary

Partitioned Output

Notice the tie!

AGENTNAME	REGION	COMMISSION	REGIONRANK
Nourhan Adel	Cairo	42,000	1
Tarek Sami	Cairo	38,500	2
Heba Younes	Cairo	38,500	2
Omar Nabil	Giza	45,200	1



Skip Behavior: The two Cairo agents tied at 38,500 both get **rank 2**. The next Cairo rank would jump directly to **4** (that's RANK's gap behavior). If you used **DENSE_RANK()**, the next rank would be 3.

— 5 PROGRESSIVE CHALLENGES

In-Class Lab Exercise

Using AcademyDB, write queries to answer each question below. Work through them in order with interactive solution reveals.

🕒 In-Class Hands-On Lab (5 Progressive Challenges)

Click to reveal SQL solutions

1. Inner Join: List every Attendance record with the StudentName and CourseName it belongs to.

Reveal Solution

2. Left Join + IS NULL: Find all Courses that have never had a single Enrollment.

Reveal Solution

3. Self Join: Find pairs of Leads that came from the same Campaign and were assigned to different Agents.

Reveal Solution

4. Subquery + IN: List SalesAgents whose AgentID appears in Leads with Status = 'Converted'.

Reveal Solution

5. Ranking: Rank Courses by total revenue (SUM of Payments.Amount) from highest to lowest, overall.

Reveal Solution

💡 Tip: Build each query one clause at a time — SELECT, then FROM, then JOIN, then WHERE — and run it often.

— CORE RELATIONAL SUMMARY

Key Takeaways

Four fundamental techniques connecting isolated database tables into actionable analytical insights.



Joins Connect Tables

INNER for matches only, LEFT/RIGHT to keep one full side, FULL OUTER to keep everything for audits.



Self, Cross & UNION

Self join compares a table to itself; CROSS JOIN builds every combination; UNION stacks result sets.



Subqueries Nest Logic

Scalar values in WHERE, lists with IN, existence checks with EXISTS, correlated for row-by-row comparisons.



Ranking Functions

ROW_NUMBER, RANK, DENSE_RANK and NTILE — with OVER() and PARTITION BY — power leaderboards and top-N reports.

Relational Thinking Completed: From isolated Excel tabs to unified multi-table enterprise intelligence.

— 4 ESSENTIAL RETENTION QUESTIONS

Quick Knowledge Check

Test your understanding of join types, subquery nuances, and window ranking behavior.

QUICK KNOWLEDGE CHECK (CLICK TO REVEAL ANSWER):

1. Which join type should you choose if you want all registered students even if they have no enrollments?

Reveal

2. What happens if a subquery inside `WHERE Price > (SELECT ...)` returns more than one row?

Reveal

3. What is the key difference between RANK() and DENSE_RANK() when two rows tie?

Reveal

4. Why is UNION ALL faster than UNION?

Reveal

Core Takeaway: Joins connect related tables, subqueries nest logical queries, and ranking functions order within groups.

INSTANT

✓ SESSION 27 COMPLETED SUCCESSFULLY

THANK YOU

Thank you for your active participation!

Next Session: Common Table Expressions (CTEs), Views & Window Aggregates

↺ Restart Presentation

Explore CTEs & Advanced SQL →